

Number: P4147R0

Date: 2026-05-12

Audience: [Evolution](#)

Target: C++29

Author: [Hana Dusíková](#)

.....

- [Introduction](#)
- [Motivation](#)
 - [Value check](#)
 - [constexpr memory leak check as a library code](#)
 - [Value transformation](#)
 - [Transformations to get constexpr non-transient allocations](#)
 - [Alternative transformations](#)
 - [Alternative transformations](#)
- [Design](#)
 - [constexpr variable](#)
 - [constexpr static variable](#)
 - [constexpr evaluation](#)
- [Free function or member function or both?](#)
 - [Argument against a free function in std:: namespace](#)
 - [Name of it](#)
 - [Why the function should be constexpr?](#)
 - [When exactly it is called?](#)
 - [Transformation on ODR-use](#)
 - [Option 2 and modules](#)
- [Implementation](#)
- [Wording](#)

P4147R0: constexpr => runtime bridge

Introduction

This paper introduces a customization point which is called when a constant-evaluated value is moving outside of its constant-evaluation (like initialization of constexpr variable, or constexpr function emitting a value to an expression / function evaluated at runtime).

Motivation

This will allow pure library solution for problems like new core wording introduced in [P3771: constexpr mutex, locks, and condition variable](#). Any library will be able provide a check just at the end of constant evaluation if value is well-formed, or a transformation like internally calling `define_static_string` to transform `std::string`'s constexpr allocation to a non-transient allocation form.

Currently there is a now way to modify or even detect a value being moved across boundaries of constant-evaluation and runtime-evaluation.

Value check

As this `magic_function()` is called when constant evaluation is left, there is no context around, but it is still part of the constant evaluation (if the function overload is `constexpr`) throwing an exception here can be used as a custom error message for when a type is moved from constant evaluation.

```
class mutex {  
    // ...  
    constexpr mutex & magic_function() const {  
        if (this->__is_locked()) {  
            throw __custom_error_message{"leaking locked mutex"};  
        }  
        return *this;  
    }  
}
```

Constexpr memory leak check as a library code

In combination with reflection one can recursively go thru all members of all reachable objects from the checked object, and check if they are heap allocated, and if so, error out. Moving [a core wording in \[expr.const\]](#) to a library code which would have a signature like this (as a free function):

```

auto && magic_function(auto && obj) {
    if constexpr (is_pointer_type(^obj)) {
        if (__is_heap_allocated(obj)) {
            throw not_released_memory(__allocation_source_location(obj));
        }
    } else if (is_lvalue_reference_type(^obj)) {
        if (__is_heap_allocated(&obj)) {
            throw not_released_memory(__allocation_source_location(&obj));
        }
    }

    // TODO: fix infinite loop in self-referencing objects
    template for (constexpr auto member: nonstatic_data_members_of(^obj, std::meta::
access_context::unchecked())) {
        // TODO: apologize to reflection folks for probably buggy example
        if constexpr (is_pointer_type(member)) {
            if (obj.[:member:] != nullptr) {
                magic_function(*obj.[:member:]);
            }
        } else if constexpr (is_lvalue_reference_type(member)) {
            magic_function(obj.[:member:]);
        }
    }

    return obj;
}

```

Value transformation

In following example transformation returns new string pointing at a static storage, instead to a heap allocation.

```

class string {
    // ...
    constexpr const char * data() const;
    constexpr size_t size() const;
    constexpr const_iterator begin() const;
    constexpr const_iterator end() const;

    consteval string magic_function() const {
        return {std::meta::define_static_string(*this), this->size()};
    }

    constexpr ~string() {
        if (!__is_static_storage(this->data())) { // assume we have such magical function
            delete[] this->data();
        }
    }
};

```

Transformations to get constexpr non-transient allocations

In following example instead of just returning same type, it changes the template argument of itself and changes its allocator, which simply won't deallocate the memory.

```

template <typename CharT, typename Traits, typename Alloc> class basic_string {
    // ...
    consteval auto magic_function() const {
        return basic_string<CharT, Traits, static_storage_alloc>{std::meta::define_static_string(*this), this->size()};
    }
}

```

Alternative transformations

If we can change type, we can go farther.

```

template <typename T> class vector {
    // ...
    consteval auto magic_function() const {
        return define_static_array(...);
    }
}

```

Alternative transformations

Perhaps most extreme example is a combination with `constexpr` coroutines, this would allow us to start a coroutine, which is immediately suspended and transfer it into runtime world, but elliding allocations. This would make such code very usable for making state machines in safety oriented industry where allocations are simply not allowed.

```
constexpr lazy_coro my_state_coroutine();

struct lazy_coro {
    struct promise_type {
        consteval auto initial_suspend() { return std::suspend_always{}; }
        consteval auto & magic_function() const {
            // create a static byte storage with similar function as define_static_object
            // copy current object into (currently impossible, but I'm exploring this)
            return // reference to the copy
        }
    };
};

static constinit auto dsm = my_state_coroutine();
```

Of course this is highly hypothetical, but it's an interesting idea.

Design

When a function is transfered from constant evaluation to a static storage or just into a value known at runtime (`constinit`, `define_static_string` and friends, or by initiating a new constant evaluation with `consteval` or immediate escalation) compiler will look for a `consteval` member function or a `consteval` free function (I personally would prefer member first, and then free one). Which takes current object by provided as R-value reference and can modify it in-place or can provide itself or any other type which is then put in its place.

constexpr variable

```
constexpr auto a = constant_calculation(); // before
constexpr auto b = magic_function(constant_calculation()); // after (implicitly)
```

constinit static variable

```
static constinit auto a = constant_calculation(); // before
static constinit auto b = magic_function(constant_calculation()); // after (implicitly)
```

consteval evaluation

```
constexpr auto constant_calculation() { ... }

auto function() { // notice it's not constexpr nor constexpr
    auto a =          constant_calculation(); // before
    auto b = magic_function(constant_calculation()); // after (implicitly)
}
```

Free function or member function or both?

I think having both would be most friendly to users. Member function limits extension of types provided by others, free functions raises question how it will be resolved and where it will be looked for.

I believe such function would be provided by users in their namespace or inside of their types quite commonly.

Argument against a free function in `std::` namespace

If we decide to have this function reside in `std::` namespace it would be a not friend customization point, user would need to close their namespace and then provide overload or even worse specialization in `std::` namespace.

Name of it

I don't particularly care, but if we are going to do a free function it shouldn't be something which already exists, maybe `operator constexpr` (as `constexpr` is combination of runtime and constant evaluation) which will give us funnily following syntax:

```
struct a_type {
    constexpr auto operator constexpr() const {
        // hey it's me, `static` :)
    }
}
```

Why the function should be `constexpr`?

It doesn't need to, this makes sure the transformation happen during compile time. If the function is not marked `constexpr` nor `constexpr` then you can't trigger compilation failure with `constexpr` exceptions. But I can imagine scenarios where there is something to be done in runtime after object is prepared in constant time, perhaps seeding a cryptography operation.

When exactly it is called?

There is multiple following options:

- whenever constant evaluation is yielding a value (immediately),
- when the value is ODR used outside constant evaluation (when used).

Both has upside and downside. Major upside for the first option the implementation is easier, whenever a `constexpr` or `constinit` variable is initialized, or in root of the any constant evaluated expression which result is accessible from runtime (not `static_assert`, but yes for template arguments).

Upside of second option is that it happens only when the value will be really visible in the runtime code. But that can also be on multiple places.

Transformation on ODR-use

```
constexpr Foo cnstxprvl = constant_calculation(); // no transformation here

consteval Foo magic_function(Foo & f) { ... }

// BEFORE author doesn't know it's going to be moved to runtime world
// AFTER transformation called as consteval code
if (magic_function(cnstxprvl)) { ...}

// and again
if (magic_function(cnstxprvl)) { ...}
```

Option 2 and modules

For the first option (immediately) there is no need to change anything, after all what you have is just a normal `constexpr` variable already initialized. But for the second option, the transformation function should be visible, which to me *seems as an argument for member functions only*.

Implementation

I have started prototyping it after Croydon meeting, but at this moment I'm only interested in EWG's opinion about usefulness of this approach.

Wording

For the first option I will probably change [\[dcl.constexpr\]](#) by adding a paragraph about the magic function being called for class types objects and same thing would be needed in [\[dcl.constinit\]](#). Interesting will be `consteval` and immediate escalation.